



AI VIET NAM – AI Conquer 2026

Trợ lý Hỏi–Đáp Tài liệu khóa học Project RAG Chatbot

Báo cáo kỹ thuật theo hướng Đi làm (Application) — Project Chatbot AI

Nhóm: **Overfitting Club**

I. Tổng quan báo cáo

Project Chatbot AI là một trợ lý hỏi–đáp tài liệu (**RAG chatbot**) cho khóa học. Báo cáo tập trung vào **phần AI cốt lõi** — truy xuất tài liệu và sinh câu trả lời bám nguồn: học viên đặt câu hỏi bằng **ngôn ngữ tự nhiên** (đa ngôn ngữ — tiếng Việt, tiếng Anh...) và nhận câu trả lời **streaming theo thời gian thực**, tổng hợp trực tiếp từ tài liệu khóa học đã nạp.

Vấn đề thực tế. Tài liệu khóa học (slide, giáo trình, ghi chú) dài hàng trăm trang; tìm kiếm từ khóa thuần không hiểu *ngữ nghĩa* câu hỏi và không trả lời trực tiếp.

Vì sao cần AI. Cần vừa *hiểu ngữ nghĩa* để tìm đúng đoạn liên quan, vừa *tổng hợp câu trả lời* bám sát nguồn (chống bịa) — đúng bài toán **Retrieval-Augmented Generation (RAG)**.

Phạm vi sản phẩm. Project chatbot là một **Django app** tự chứa *toàn bộ* vòng đời RAG:

- **Ingest** (xử lý nền, bất đồng bộ): trích text/OCR → chunk → embed → index vào OpenSearch — chạy nền bằng Celery; bước phụ còn đẩy toàn văn vào **đồ thị tri thức LightRAG** (đã hiện thực, cấu hình bật, fail-safe; chỉ chạy khi có hạ tầng PostgreSQL/Neo4j).
- **Answering** (thời gian thực): truy xuất theo *hai đường bổ trợ* — (i) *hybrid* BM25 + kNN hợp nhất bằng RRF rồi lọc sán cosine trên OpenSearch (đường chính; rerank cross-encoder đã hiện thực, tùy chọn — chưa bật trong cấu hình hiện tại), và (ii) *đồ thị tri thức* LightRAG [1] truy hồi thực thể/quan hệ cho câu hỏi suy luận đa bước (đã hiện thực, cấu hình bật, fail-safe; vận hành khi có hạ tầng KG) — rồi sinh câu trả lời bằng **LLM Gemini điều phối qua Google ADK**, trả về **SSE streaming**.

Project chatbot được tổ chức theo nhiều lớp rõ ràng (giao tiếp HTTP → điều phối service → lõi RAG → nền Celery + dữ liệu). Các thành phần *bên ngoài* mà project này phụ thuộc — OpenSearch (kho vector+BM25), S3 (file gốc), Gemini (LLM/embedding), Redis (Celery), MariaDB (đọc bản ghi tài liệu) — chỉ được nhắc ở mức **ranh giới phụ thuộc**, không phải trọng tâm báo cáo.

Một câu định vị sản phẩm

Project chatbot là một trợ lý hỏi–đáp tài liệu theo kiến trúc RAG: nó nạp tài liệu khóa học thành kho tri thức tìm kiếm được, rồi với mỗi câu hỏi sẽ truy xuất các đoạn liên quan từ OpenSearch (và bổ sung từ đồ thị tri thức LightRAG khi câu hỏi cần quan hệ và hạ tầng KG sẵn sàng) rồi sinh câu trả lời bám nguồn, trả về người dùng dưới dạng SSE streaming.

Mục lục

I.	Tổng quan báo cáo	1
II.	Định nghĩa bài toán	4
III.	Dữ liệu sử dụng	4
IV.	Tiền xử lý và chuẩn bị dữ liệu	5
V.	Phương pháp và mô hình sử dụng	6
V.1.	Truy xuất hybrid (BM25 + kNN) và RRF	6
V.2.	Truy xuất đồ thị tri thức (LightRAG, đã hiện thực, cấu hình bật)	7
V.3.	Sinh câu trả lời bằng Google ADK	8
VI.	Kiến trúc project chatbot	8
VII.	Luồng huấn luyện và suy luận	10
VII.1.	Luồng “huấn luyện” = Ingest (xử lý nền)	10
VII.2.	Luồng suy luận = Chat (thời gian thực, SSE)	10
VIII.	Công cụ và công nghệ sử dụng	12
IX.	Cấu trúc mã nguồn và hướng dẫn chạy	13
X.	Thực nghiệm và đánh giá mô hình	14
X.1.	Đánh giá LightRAG/KG riêng (A/B trên câu hỏi quan hệ)	16
XI.	Phân tích lỗi và hạn chế	17
XII.	Kết luận và hướng phát triển	18
XIII.	Tài liệu tham khảo	20
	Phụ lục	21

II. Định nghĩa bài toán

Loại bài toán. *Retrieval-Augmented Question Answering* (RAG) trên kho tài liệu khóa học.

Đầu vào. Câu hỏi bằng ngôn ngữ tự nhiên (đa ngôn ngữ, chủ yếu tiếng Việt/tiếng Anh); kèm `conversation_id` (tùy chọn, giữ ngữ cảnh) và `media_ids` (tùy chọn, đính kèm tài liệu vào lượt hỏi).

Đầu ra. Câu trả lời văn bản **stream qua SSE**, dựa trên tài liệu đã nạp, kèm **trích nguồn** (tên tài liệu, số trang, điểm liên quan).

Hai luồng con. *Ingest* (biến tài liệu thô thành kho tri thức) và *Answering* (truy xuất đoạn văn từ OpenSearch, bổ sung thực thể/quan hệ từ đồ thị tri thức LightRAG khi câu hỏi cần quan hệ và hạ tầng KG sẵn sàng, rồi sinh câu trả lời).

Tiêu chí thành công.

- Truy xuất đúng đoạn liên quan.
- Câu trả lời *bám nguồn* (groundedness/faithfulness).
- Độ trễ chấp nhận được, đặc biệt **time-to-first-token** do giao diện streaming.
- Demo hoàn chỉnh từ nạp tài liệu đến nhận câu trả lời.

Định nghĩa bài toán

Bài toán được mô hình hóa là retrieval-augmented question answering: đầu vào là câu hỏi bằng ngôn ngữ tự nhiên (đa ngôn ngữ), module truy xuất các đoạn tài liệu liên quan từ kho tri thức (OpenSearch, và đồ thị tri thức LightRAG khi câu hỏi cần quan hệ và hạ tầng KG sẵn sàng) rồi sinh câu trả lời bám sát nguồn, trả về dưới dạng SSE streaming.

III. Dữ liệu sử dụng

Nguồn. Tài liệu khóa học định dạng **PDF** (kể cả PDF scan/ảnh cần OCR) và **Word (DOCX)**. Module đọc danh sách tài liệu cần xử lý từ bảng `chatbot_documents` và `media` (`model_managed=False` — module không sở hữu schema, chỉ ánh xạ để đọc).

Lưu trữ.

- **File gốc:** trên **S3** (tương thích MinIO), tải bằng `boto3`.
- **Kho tri thức** (OpenSearch là chính; đồ thị tri thức là thành phần bổ trợ, chỉ vận hành khi có hạ tầng PostgreSQL/Neo4j):
 - **OpenSearch 2.19.1** (chính) — chỉ mục `rag-index` (vector kNN + text BM25), phụ trợ `document-summary-index` và `chatbot-chat-history` (trí nhớ dài hạn).

- **LightRAG Knowledge Graph** [1] (app/lightrag/, hạ tầng PostgreSQL/Neo4j) — lưu *thực thể & quan hệ* cho truy hồi đa bước, **cấu hình bật** (LIGHTRAG_ENABLED=true), fail-safe; chỉ vận hành/đo được khi có hạ tầng PostgreSQL/Neo4j [2] + pgvector [3].

Quy mô (bộ dữ liệu thực nghiệm hiện tại). 9 tài liệu khóa học AIO 2026 — Python cơ bản (vòng lặp; rẽ nhánh & chatbot theo luật), SQL & ERD, Coding Methodology, Image & Video Data, Teamwork Tools (Agile/Scrum/Jira), hướng dẫn làm video, và chính sách bảo mật — nạp thành **782 chunk con** (cùng 9 bản ghi cha) trên rag-index, khoảng 33–101 trang mỗi tài liệu. Mỗi tài liệu sinh 1 bản ghi cha + N bản ghi con tương ứng N chunk.

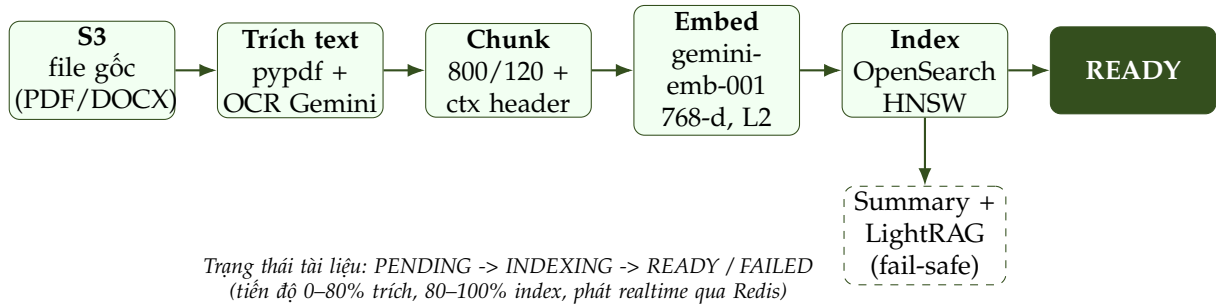
Vấn đề dữ liệu. Trang scan chất lượng thấp; bảng/công thức khó OCR; tài liệu dài không đều; tiếng Việt có dấu; nội dung trùng lặp. Được xử lý một phần ở khâu tiền xử lý (mục IV.) và phân tích ở mục XI..

IV. Tiền xử lý và chuẩn bị dữ liệu

Khâu quyết định chất lượng RAG, được điều phối bởi app/pipelines/ingest.py (hàm run_ingest_pipeline) và kích hoạt nền qua app/tasks/ingest.py (Celery, broker Redis). Các bước:

- 1) Tải file gốc.** Tải bytes file gốc từ S3 theo media.file_name (S3Client ở base tự đọc env AWS_S3_*).
- 2) Trích text theo trang.** pypdf cho trang có text-số; trang ít hơn 20 ký tự hoặc có ảnh ≥ 5000 byte thì **OCR cả trang bằng Gemini** (fail-safe: OCR xong thì giữ text pypdf). File Word chuyển PDF trước bằng **LibreOffice** (soffice, timeout 120s).
- 3) Chunk theo trang + contextual header.** RecursiveCharacterTextSplitter [4] (800/120); mỗi chunk gắn header giàu ngữ cảnh ngay vào text để chunk lẻ tự đủ ngữ cảnh khi truy hồi → tăng recall.
- 4) Embedding.** gemini-embedding-001 **768 chiều** (Matryoshka/MRL từ 3072), **tự chuẩn hóa L2** (vector <3072 chiều không được normalize sẵn); task_type=RETRIEVAL_DOCUMENT; batch 50.
- 5) Index parent-child (OpenSearch HNSW).** 1 cha (metadata) + N con (chunk_text, chunk_vector, số trang) nối qua join_doc_join; write-then-delete để không có khoảng trống chỉ mục khi tải nạp.
- 6) Phụ (fail-safe).** Index *tóm tắt* (≤ 200 ký tự) vào document-summary-index; **LightRAG/KG** [1] (đã hiện thực, cấu hình bật, fail-safe; chỉ chạy khi có hạ tầng PostgreSQL/Neo4j) — khi đó LightRAG nhận *toàn văn* tài liệu, dùng LLM gemini-2.5-flash trích *thực thể & quan hệ* vào KG (PostgreSQL/Neo4j), idempotent theo khóa chatbot_{document_id} (xóa-rồi-chèn), chạy SAU khi rag-index OpenSearch đã xong. Lỗi bước phụ **không** chặn trạng thái READY.

Máy trạng thái tài liệu (app/enums/document_status.py): PENDING → INDEXING → READY hoặc FAILED (lỗi ngoại vi S3/Gemini/OpenSearch). Tiến độ indexed_percent (0–100) phát realtime qua Redis (khâu trích 0–80%, khâu index 80–100%). Sơ đồ ở Hình 1.



Hình 1: Pipeline Ingest của module (app/pipelines/ingest.py): S3 → trích text (pypdf/OCR Gemini) → chunk + contextual header → embed (768-d) → index parent-child vào OpenSearch (HNSW); nhánh phụ (fail-safe) index tóm tắt và **LightRAG/KG** (cấu hình bật, vận hành khi có hạ tầng KG) nhưng **không** ảnh hưởng trạng thái READY.

V. Phương pháp và mô hình sử dụng

Phần “AI” cốt lõi của module gồm: (1) **truy xuất hybrid** BM25 + kNN trên OpenSearch (app/rag, app/opensearch) — đường truy xuất *chính*, luôn bật; (2) **truy xuất đồ thị tri thức** LightRAG [1] (app/lightrag) — đường truy xuất *phụ* đã hiện thực, cấu hình bật, fail-safe (vận hành khi có hạ tầng KG); và (3) **sinh câu trả lời bằng agent qua Google ADK** (app/adk, app/tools) tổng hợp ngữ cảnh từ đường hybrid (và bổ sung từ đường đồ thị khi câu hỏi cần quan hệ).

V.1. Truy xuất hybrid (BM25 + kNN) và RRF

Baseline. Tìm kiếm từ khóa thuần **BM25** (khớp trên chunk_text).

Truy xuất hybrid (app/opensearch/retriever.py). Kết hợp **BM25** (khớp từ khóa) và **kNN vector** (khớp ngữ nghĩa) trên cùng kho OpenSearch, rồi *hợp nhất* hai danh sách kết quả bằng **Reciprocal Rank Fusion (RRF)** — ưu tiên những đoạn được cả hai cách xếp hạng cao (cấu hình RRF_K = 60). Câu hỏi cũng được embed và chuẩn hóa trước khi tìm; vector chạy trên chỉ mục **HNSW** của OpenSearch [5].

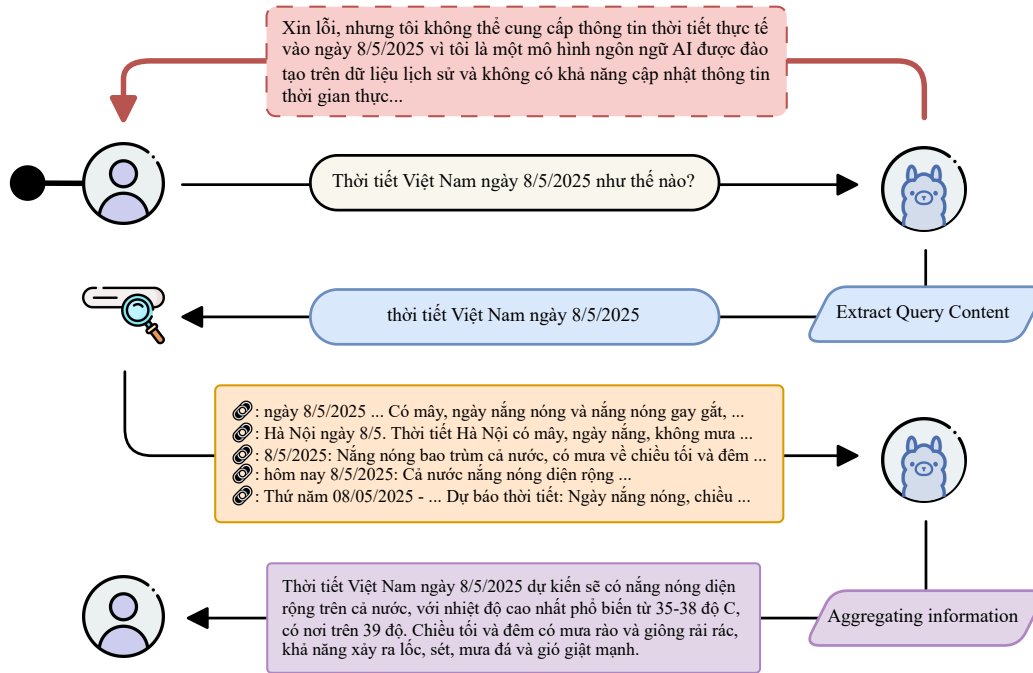
Rerank (app/rag/reranker_client.py). Tốp ứng viên (TOP_N = 30) được rerank bằng **cross-encoder** bge-reranker-v2-m3 qua một HTTP endpoint cắm ngoài (timeout 15s), lọc theo RERANK_SCORE_THRESHOLD = 0.3, trả TOP_K = 5 đoạn. Là cơ chế *tùy chọn, fail-safe*: nếu endpoint rerank để trống thì giữ nguyên thứ tự hybrid-RRF, có thể lọc bằng ngưỡng cosine tối thiểu (HYBRID_MIN_COSINE; mặc định code 0.0, .env.example gợi ý 0.80) như tín hiệu chống bịa. Việc lọc nhiều tầng nằm ở app/tools/knowledge_base.py.

Lưu ý về phạm vi

Query rewriting đã được *khai báo* trong cấu hình (QUERY_REWRITE_ENABLED, QUERY_REWRITE_MAX_VARIANTS=3) nhưng **chưa được hiện thực trong code** ở thời điểm báo cáo — không có tham chiếu trong luồng truy xuất. Vì vậy trình bày như *hướng phát*

triển (mục XII.), không phải tính năng đang chạy. Tương tự, TOP_K/TOP_N/RRF_K được *hardcode* trong app/rag/config.py (5/30/60).

Pipeline truy xuất: *embed query* (L2) → *hybrid* (BM25 + kNN, mỗi nhánh TOP_N) → *RRF* ($k = 60$) → *rerank* (tùy chọn) → *lọc ngưỡng* → trả TOP_K (Hình 2). Đây là đường truy xuất *chính*; đường thứ hai (đồ thị tri thức LightRAG) trình bày ở mục V.2. kế tiếp.



Hình 2: Minh họa pipeline truy xuất (tham khảo phong cách từ bộ kit). Trong module, hybrid BM25 + kNN được hợp nhất bằng RRF rồi lọc sàn cosine; rerank cross-encoder là bước tùy chọn (đã hiện thực, chưa bật do thiếu endpoint).

V.2. Truy xuất đồ thị tri thức (LightRAG, đã hiện thực, cấu hình bật)

Ngoài truy xuất hybrid trên OpenSearch, module xây thêm một đường truy xuất *đồ thị tri thức* bằng **LightRAG** [1] (app/lightrag/) theo tinh thần **GraphRAG** [6] — *cấu hình bật* (LIGHTRAG_ENABLED=true), thiết kế chạy bổ trợ cho đường hybrid và *fail-safe* (mọi lỗi được nuốt/log, không bao giờ đẩy tài liệu sang FAILED); chỉ thực sự truy hồi khi có hạ tầng PostgreSQL/Neo4j + pgvector và khi agent quyết định gọi.

Lúc ingest (LightRagIndexer). Toàn văn tài liệu được đưa vào LightRAG; một LLM **trích thực thể/quan hệ** (gemini-2.5-flash) dựng đồ thị, lưu *đồ thị* vào Neo4j và KV/vector/trạng thái vào PostgreSQL. Idempotent theo doc_id = chatbot_{document_id} (xóa-rồi-chèn) nên tải nạp không trùng.

Lúc chat (LightRagQuerier). Truy vấn ở **mode mix** — *hợp nhất* truy xuất vector (đoạn văn) và truy xuất đồ thị (thực thể + quan hệ); đặt only_need_context=True để LightRAG chỉ trả *ngữ*

cảnh thô (entities/reasons + đoạn liên quan), **không tự sinh câu trả lời** — việc tổng hợp dành cho agent ADK (mục V.3.). Vì kết quả KG không kèm metadata chunk nên *không sinh citation*.

Vì sao cần đồ thị. BM25 + kNN mạnh ở tra cứu sự kiện đơn lẻ nhưng yếu ở câu hỏi *quan hệ* (“X ảnh hưởng Y thế nào”, “cái gì liên kết A với B”) và suy luận nhiều bước bắc qua nhiều tài liệu; đồ thị thực thể/quan hệ bù đúng điểm yếu này. Hạ tầng PostgreSQL/Neo4j nằm sau compose profile lightrag, chỉ bật khi cần.

V.3. Sinh câu trả lời bằng Google ADK

Câu trả lời được sinh bởi LLM Gemini (gemini-2.5-flash) điều phối qua **Google ADK** [7] (app/adk/):

- **Agent** aio_chatbot (app/adk/agent.py, stateless, cache bằng lru_cache) trang bị các tool: search_knowledge_base (truy hồi đoạn văn từ OpenSearch hybrid), search_knowledge_graph (truy hồi *thực thể/quan hệ* từ LightRAG KG — mode mix, fail-safe; chỉ truy hồi khi cờ bật và hạ tầng KG sẵn sàng), get_document_url, create_mind_map (app/adk/tools.py, app/tools/). Cả hai tool truy hồi *luôn được đăng ký* dù KG bật hay tắt; instruction phân vai rõ — search_knowledge_base cho tra cứu sự kiện, search_knowledge_graph cho câu hỏi quan hệ/đa bước — cờ LIGHTRAG_ENABLED bật/tắt KG được xử lý *lúc chạy* trong LightRagQuerier, không phải bằng đăng ký tool.
- **Runner** (app/adk/runner.py) chạy StreamingMode.SSE; InMemorySessionService phân vùng phiên theo session_id.
- **Planner**: BuiltInPlanner với ThinkingConfig(include_thoughts=True) cho Gemini 2.5+ (phát sự kiện *thinking*). app/adk/stream_handler.py chống lặp text khi stream.

Trí nhớ & ngữ cảnh (app/chat_pipeline/). Mỗi lượt nạp: (i) **lịch sử N lượt** gần nhất (CHAT_HISTORY_SIZE = 10); (ii) **trí nhớ dài hạn (LTM)** — truy xuất CHAT_LTM_TOP_K = 3 hội thoại cũ liên quan từ chatbot-chat-history (kNN 768-d, ngưỡng CHAT_LTM_MIN_SCORE = 0.5); (iii) ngữ cảnh RAG mặc định CHAT_CONTEXT_TOP_K = 5.

Lý do chọn. Pretrained LLM mạnh, *không cần train*; hybrid + RRF (kèm sán cosine; rerank cross-encoder tùy chọn khi bật) cân bằng recall/precision; ADK chuẩn hóa vòng đời agent/tool/streaming; embedding bất đối xứng + L2 phù hợp truy xuất ngữ nghĩa đa ngôn ngữ. Bổ sung **LightRAG/GraphRAG** [1], [6] làm đường truy xuất *bổ trợ* (đã hiện thực, cấu hình bật) vì truy xuất đoạn văn thuần yếu ở câu hỏi quan hệ/đa bước; đổi lại phải dựng thêm hạ tầng PostgreSQL/Neo4j và chịu chi phí LLM trích thực thể lúc ingest — chấp nhận được để có khả năng suy luận đa tài liệu. (Module hỗ trợ thay LLM/embedding sang Ollama qua LiteLlm — LLM_PROVIDER=ollama.)

VI. Kiến trúc project chatbot

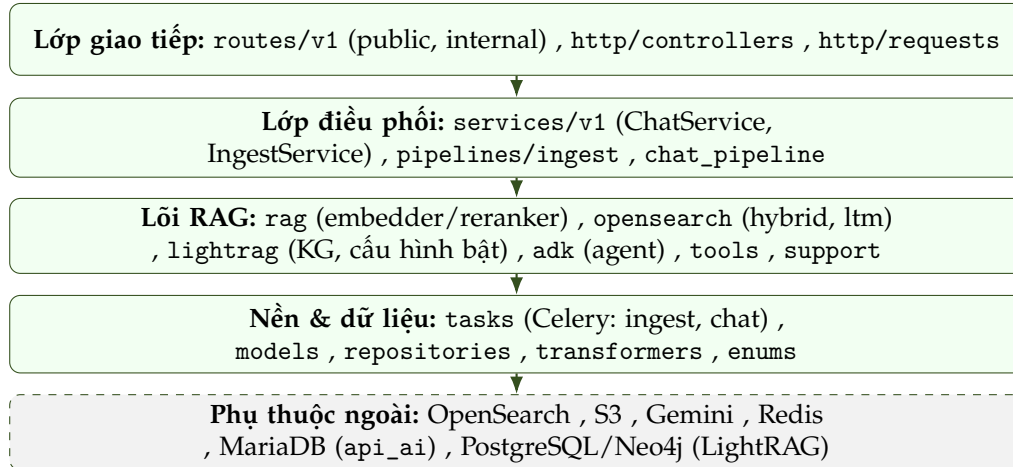
Project chatbot là một Django [8] app tổ chức theo **nhiều lớp** (Bảng 1, Hình 3): lớp giao tiếp HTTP → lớp điều phối service → lõi RAG → nền Celery [9] + lớp dữ liệu. Cách tách lớp này

giúp luồng *ingest* và *chat* dùng chung lõi RAG (embed, truy xuất, OpenSearch) nhưng tách biệt phần điều phối.

Bảng 1: Các package chính của app/ trong project chatbot.

Package	Vai trò
<code>routes/, http/</code>	Định tuyến (public + internal) và controller/request DRF (JSON-only)
<code>services/v1/</code>	Điều phối nghiệp vụ: ChatService (prepare/stream), IngestService
<code>pipelines/</code>	<code>run_ingest_pipeline</code> : dàn nhạc toàn bộ luồng ingest
<code>chat_pipeline/</code>	Cấu hình chat, nạp lịch sử, LTM, đóng gói sự kiện SSE
<code>rag/</code>	Embedder (Gemini/Ollama), reranker client, cấu hình truy xuất
<code>opensearch/</code>	Indexer, retriever (hybrid+RRF), summary, LTM, client
<code>lightrag/</code>	LightRAG Knowledge Graph: thực thể/quan hệ (cấu hình bật, fail-safe; vận hành khi có hạ tầng KG)
<code>adk/</code>	Agent, Runner, tool, stream handler (Google ADK)
<code>tools/</code>	Tool nghiệp vụ: <code>knowledge_base</code> (lọc), <code>mind_map</code>
<code>support/</code>	Trích text/OCR, chunk, contextual header, Word→PDF
<code>tasks/</code>	Celery: <code>ingest_document</code> , hậu lượt chat (tiêu đề, LTM)
<code>models/, repositories/, transformers/, enums/</code>	ORM (managed=False), truy cập dữ liệu, biến đổi response, hằng trạng thái

Ranh giới phụ thuộc ngoài. Module gọi tới (qua client dùng chung): **OpenSearch** (opensearch-py), **S3** (boto3), **Gemini** (google-genai), **Redis** (Celery + realtime), và đọc bảng tài liệu trong **MariaDB** (managed=False); **LightRAG** (cấu hình bật; chỉ vận hành khi cung cấp hạ tầng) dùng thêm **PostgreSQL/Neo4j** làm hạ tầng đồ thị tri thức. Các thành phần này nằm *ngoài* module và không phải trọng tâm báo cáo.



Hình 3: Kiến trúc nội bộ project chatbot theo lớp: giao tiếp HTTP → điều phối service → lỗi RAG → nền Celery + dữ liệu → phụ thuộc ngoài.

VII. Luồng huấn luyện và suy luận

RAG không huấn luyện model mới, nên hai luồng được diễn giải lại: “huấn luyện” = **Ingest** (xây tri thức, chạy nền bất đồng bộ); “suy luận” = **Chat** (thời gian thực, SSE).

VII.1. Luồng “huấn luyện” = Ingest (xử lý nền)

Điểm vào của module là route nội bộ POST `/api/internal/v1/chatbot/documents/ingest` (`routes/v1/internal.py`) — nhận `document_id`, đẩy tác vụ Celery `chatbot.ingest_document` (trả 202) → `run_ingest_pipeline` thực thi: trích text/OCR → chunk + embed → index OpenSearch → READY. Sau khi index OpenSearch thành công, pipeline còn chạy *bước phụ fail-safe*: index tóm tắt và đồ thị tri thức **LightRAG** [1] (`LightRagIndexer.index_document`, xóa-rồi-chèn theo `doc_id=chatbot_{document_id}`); bước này *cấu hình bật* (chỉ chạy khi có hạ tầng PostgreSQL/Neo4j) và mọi lỗi đều bị nuốt — không bao giờ làm tài liệu chuyển FAILED. Chi tiết và sơ đồ ở mục IV. và Hình 1.

VII.2. Luồng suy luận = Chat (thời gian thực, SSE)

Điều phối bởi `app/services/v1/chat_service.py`:

1. FE gọi POST `/api/v1/chatbot/chat` (`routes/v1/public.py`).
2. **prepare** (đồng bộ): tạo/lấy conversation; khóa bằng `SELECT...FOR UPDATE` — nếu đã có tin nhắn `PROCESSING` thì trả **409 Conflict**; lưu message user + tạo *placeholder* message bot (`status = PROCESSING`).
3. **stream**: nạp *LTM* + *lịch sử* vào phiên ADK → Runner chạy Agent → Agent gọi `search_knowledge_base` (hybrid + RRF + sản cosine; rerank cross-encoder tùy chọn) cho tra cứu đoạn văn, và *tùy ngữ cảnh* gọi `search_knowledge_graph` (LightRAG KG, mode

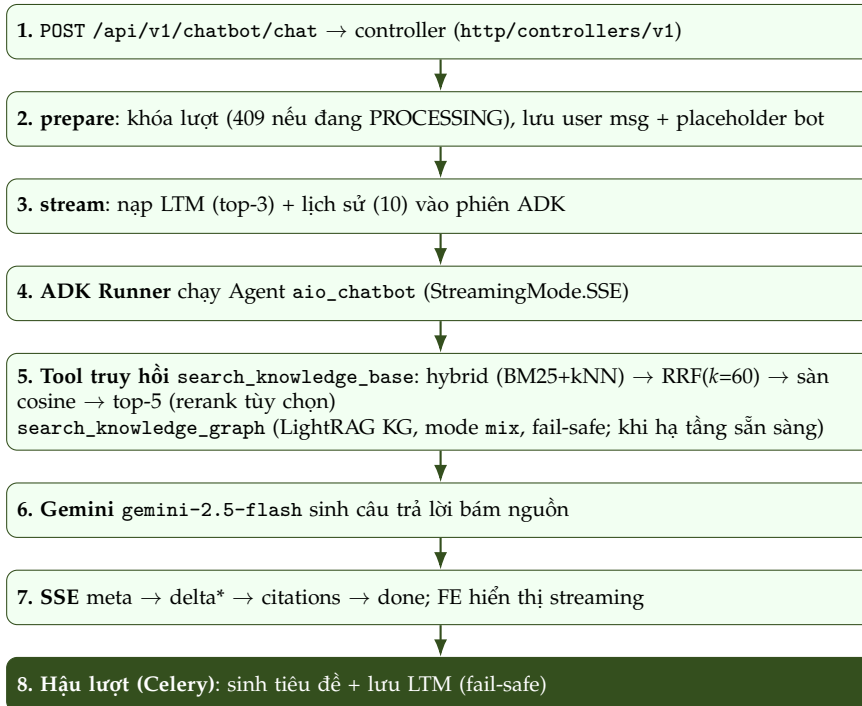
mix, fail-safe; chỉ truy hồi khi hạ tầng KG sẵn sàng) cho câu hỏi suy luận đa bước → Gemini sinh câu trả lời → **stream** từng mẫu.

4. **Giao thức SSE** (app/chat_pipeline/sse.py): meta → (delta | thinking | citations)* → mindmap_pending? → mindmap? → done | error. Wire format data: <json>, Unicode giữ nguyên (ensure_ascii=False), tắt buffering (X-Accel-Buffering: no).
5. **Hậu lượt** (Celery, app/tasks/chat.py, fail-safe): sinh *tiêu đề hội thoại* (gemini-2.5-flash) và lưu *LTM* (vector 768-d). Trạng thái message: PROCESSING → SUCCESS / ERROR.

Giao thức SSE thực tế (text/event-stream)

```
data: {"type":"meta","conversation_id":12,"message_id":85,"citations":[]}
data: {"type":"thinking","content":"..."}          # khi Gemini 2.5 bắt thinking
data: {"type":"delta","content":"doan van ban tra loi ..."}
data: {"type":"delta","content":"... tiếp theo ..."}
data: {"type":"citations","citations":[{"original_name":"...","page":7,"score":0.82}]}
data: {"type":"mindmap_pending"}                    # nếu user yêu cầu sơ đồ tư duy
data: {"type":"mindmap","mind_map":{"title":"...","nodes":[]}}
data: {"type":"done","status":"success","message_id":85,"total_tokens":1234}
```

Toàn bộ luồng suy luận được tóm tắt ở Hình 4.



Hình 4: Luồng Chat/Inference (SSE) bên trong module: từ request, qua ChatService (prepare/stream), nạp ngữ cảnh, Agent ADK gọi tool truy xuất hybrid, Gemini sinh câu trả lời và stream về FE, cuối cùng là tác vụ hậu lượt.

VIII. Công cụ và công nghệ sử dụng

Bảng 2 liệt kê stack của module (lấy từ `requirements.txt`). Một số công nghệ then chốt:

- **OpenSearch** [5] (`opensearch-py`): lưu vector kNN/HNSW và BM25 trong cùng store → hybrid retrieval dễ, không cần vector DB riêng.
- **Google ADK** [7] (`google-adk`) + **Gemini** [10] (`google-genai`): chuẩn hóa agent/tool/streaming; Gemini lo OCR, embedding, summary, chat.
- **Celery + Redis** [9]: chạy ingest nặng ở chế độ nền, không chặn request.
- **langchain-text-splitters** (`RecursiveCharacterTextSplitter`), **pypdf** (trích text + số trang), **boto3** (S3), **LibreOffice** (Word→PDF).

Bảng 2: Tech-stack của project chatbot (từ `requirements.txt`).

Thành phần	Phiên bản	Dùng cho
Django + DRF	5.2 / 3.15+	Web framework, REST API (JSON-only)
Celery + Redis	5.4 / 5.0	Hàng đợi tác vụ ingest & hậu lượt chat
google-genai (Gemini)	1.x	OCR, embedding, summary, sinh câu trả lời
google-adk	1.x	Agent + tool RAG + Runner streaming SSE
opensearch-py	2.4+	Index/truy xuất hybrid (kNN/HNSW + BM25)
langchain-text-splitters	0.2+	Chunk văn bản (<code>RecursiveCharacterTextSplitter</code>)
pypdf	4.x	Trích text PDF theo trang
boto3	1.34+	Tải file gốc từ S3/MinIO
litellm	1.50+	Backend Ollama cho ADK (<code>LLM_PROVIDER=ollama</code>)
lightrag-hku, neo4j, asyncpg	–	Knowledge Graph (cấu hình bật, fail-safe; vận hành khi có hạ tầng)

Bảng 3: Endpoint của project chatbot (routes/v1).

Method	Path	Mô tả
POST	/api/v1/chatbot/chat	Hỏi–đáp RAG, trả SSE
GET	/api/v1/chatbot/conversations	Danh sách hội thoại (phân trang, tìm kiếm)
PATCH/DELETE	/api/v1/chatbot/conversations/{id}	Đổi tên / xóa mềm hội thoại
GET	/api/v1/chatbot/conversations/{id}/messages	Danh sách tin nhắn
POST	/api/internal/v1/chatbot/documents/ingest	(internal) Enqueue ingest → 202
POST	/api/internal/v1/chatbot/documents/purge	(internal) Xóa tài liệu khỏi kho

IX. Cấu trúc mã nguồn và hướng dẫn chạy

Cây thư mục module (modules/chatbot):

Cấu trúc project chatbot

```
modules/chatbot/
|-- app/
|   |-- routes/ http/      # dinh tuyen + controller/request (DRF)
|   |-- services/v1/      # ChatService, IngestService
|   |-- pipelines/        # run_ingest_pipeline (dan nhac ingest)
|   |-- chat_pipeline/    # config, history, ltm, sse
|   |-- rag/              # embedder, reranker_client, config
|   |-- opensearch/       # indexer, retriever, summary, ltm, client
|   |-- adk/              # agent, runner, tools, stream_handler
|   |-- tools/            # knowledge_base, mind_map
|   |-- support/          # extract/OCR, page_chunker, word_to_pdf
|   |-- tasks/            # celery: ingest, chat (hau luot)
|   |-- models/ repositories/ transformers/ enums/
|   '-- lightrag/         # Knowledge Graph (cau hinh bat, fail-safe; chay khi co ha
tang)
|-- database/migrations/
'-- routes/v1/            # public.py, internal.py
```

Điều kiện chạy (phụ thuộc ngoài). Module nằm trong dịch vụ Django; cần sẵn: **OpenSearch 2.19.1** (<http://opensearch:9200>), **Redis** (Celery), khóa **Gemini**, và **MariaDB** chứa bảng tài liệu.

Chạy module (dịch vụ Django + Celery worker)

```
# Bien moi truong then chot (khong commit secret)
export GEMINI_API_KEY=... # cho OCR/embed/chat
export OPENSEARCH_URL=http://opensearch:9200
export OPENSEARCH_INDEX=rag-index
export CELERY_BROKER_URL=redis://redis:6379/0

# Web (phuc vu /api/v1/chatbot/*) + worker (ingest + hau luot chat)
python manage.py runserver 0.0.0.0:8000
celery -A config worker -l info
```

Tham số then chốt (giá trị thật). CHUNK_SIZE=800, CHUNK_OVERLAP=120, OPENSEARCH_VECTOR_DIMS=768, TOP_K=5, TOP_N=30, RRF_K=60 (hardcode trong app/rag/config.py), CHAT_HISTORY_SIZE=10, CHAT_LTM_TOP_K=3, model gemini-2.5-flash / gemini-embedding-001.

X. Thực nghiệm và đánh giá mô hình

Đánh giá RAG theo **3 tầng** (truy xuất / sinh / hệ thống) — tách riêng để biết lỗi nằm ở *lấy nhầm tài liệu* hay *lấy đúng mà trả lời sai*. Với mỗi câu hỏi, nhóm gán trước các đoạn nguồn *đúng* (golden) và so với *top-5* đoạn hệ trả về; một đoạn được tính là đúng khi trùng tài liệu (và trùng trang nếu golden có chỉ định). Mỗi chỉ số tính cho từng câu rồi lấy trung bình toàn bộ (số liệu ở Bảng 4 và Bảng 5).

Tầng truy xuất (cần *golden* đoạn liên quan):

- **Hit@5** — có lấy được ít nhất một đoạn đúng trong top-5 hay không (chỉ Có/Không).
- **Recall@5** — trong các đoạn đáng ra phải lấy, hệ lấy được bao nhiêu phần (sợ bỏ sót).
- **Precision@5** — trong 5 đoạn lấy ra, bao nhiêu đoạn thật sự hữu ích (sợ lấy thừa).

Tầng sinh câu trả lời — chấm bằng **LLM-as-judge** (gemini-2.5-flash chấm câu trả lời) và so với đáp án mẫu:

- **Faithfulness** (bám nguồn) — câu trả lời có *bám đúng tài liệu lấy về* không, hay nói thêm điều không có trong nguồn (chống *bịa*).
- **Answer Relevance** — câu trả lời có *trả lời trùng câu hỏi* không.
- **Correctness** — câu trả lời *khớp nghĩa* với đáp án mẫu đến mức nào.

Tầng hệ thống:

- **TTFT** (time-to-first-token) — thời gian đến khi *mẫu chữ đầu tiên* hiện ra; quan trọng với streaming SSE.
- **Latency** — tổng thời gian đến khi trả lời *xong*; báo theo **p50** (ca điển hình) và **p95** (lúc tệ).

- **Tỉ lệ thành công** — bao nhiêu câu hệ trả lời được, không lỗi.
- **Thời gian ingest** — thời gian xử lý 1 tài liệu (tải + trích/OCR + chunk + embed + index), đo trên *mẫu vài tài liệu*.

Cách tạo test set. Rút câu hỏi từ tài liệu thật, gán đoạn nguồn đúng (golden) theo (document_id, trang), viết đáp án mẫu; chạy mỗi cấu hình rồi đo.

Thiết lập thực nghiệm đã chạy

Bộ golden QA do nhóm tự tạo: **28 câu hỏi** (đa ngôn ngữ) rút từ **9 tài liệu** khóa học thật (782 chunk), mỗi câu gán đoạn nguồn đúng theo (document_id, trang) và một đáp án mẫu. Tầng truy xuất chấm tự động (Recall@5/Precision@5/Hit@5) qua management command eval_rag; tầng sinh chạy *agent ADK thật* rồi chấm Faithfulness/Answer-Relevance/Correctness bằng **LLM-as-judge** (gemini-2.5-flash) và đo thời gian phản hồi. Mỗi cấu hình chạy 1 lần trên cùng bộ câu hỏi (đường truy hồi KG tắt để đo đúng đường hybrid).

Bảng 4: **Tầng truy xuất** trên 28 câu golden QA (9 tài liệu, 782 chunk), so 3 cấu hình. “Triển khai” là đường search() thực tế (hybrid BM25+kNN + RRF + sà cosine 0.80; cross-encoder rerank cấm sẵn nhưng *không* chạy khi đo do endpoint để trống).

Cấu hình	Recall@5	Precision@5	Hit@5
Baseline (BM25 thuần)	0.411	0.093	0.464
Hybrid (BM25+kNN+RRF)	0.821	0.200	0.857
Triển khai (hybrid+RRF+sà cosine)	0.875	0.214	0.893

Bảng 5: **Tầng sinh + hệ thống** ở cấu hình triển khai (28 câu). Faithfulness/Answer-Relevance/Correctness chấm bằng **LLM-as-judge** (gemini-2.5-flash).

Nhóm / Chỉ số	Giá trị
<i>Sinh — LLM-as-judge (thang 0–1)</i>	
Faithfulness (bám nguồn, chống bịa)	1.000
Answer Relevance (trúng câu hỏi)	0.993
Correctness (so đáp án mẫu)	0.929
<i>Hệ thống</i>	
TTFT (p50 / p95)	5.15 / 5.82 s
Latency (p50 / p95)	5.36 / 5.97 s
Tỉ lệ thành công	28/28 (100%)
Thời gian ingest (mẫu)	7.5 s (text) – 255 s (PDF cần OCR)

Nhận xét. (1) *Hybrid* vượt xa BM25 thuần (Recall@5 0.411 → 0.821; Hit@5 0.464 → 0.857) — định lượng giá trị của kNN ngữ nghĩa + RRF cho câu hỏi diễn đạt lại (paraphrase), không trùng từ khóa. (2) Cấu hình triển khai thực tế (đường search(): hybrid + RRF + sản cosine + chọn top-*k*) nhỉnh hơn hybrid thô (Recall +0.054, Hit +0.036); lưu ý phần cải thiện này *không* đến từ cross-encoder rerank (chưa chạy do thiếu endpoint) mà từ sản cosine + chọn top-*k*. (3) **Hit@5** = 0.893: ≈89% câu có ít nhất một đoạn đúng trong top-5. (4) Tầng sinh: **Faithfulness** 1.0, **Answer-Relevance** 0.993, **Correctness** 0.929 cho thấy câu trả lời bám nguồn tốt, trúng câu hỏi và khớp đáp án mẫu. Tổng thể hệ mạnh ở *tra cứu sự kiện/định nghĩa*, còn yếu ở *suy luận nhiều bước, câu hỏi mơ hồ* — đúng nhóm câu mà đường **LightRAG/KG** (search_knowledge_graph, mode mix) hướng tới hỗ trợ.

Lưu ý trung thực. Test set nhỏ ($n = 28$) và golden do nhóm tự soạn từ tài liệu thật nên số mang tính *minh họa có cơ sở*, chưa phải bằng chứng thống kê mạnh. **Precision@5** (0.214) bị chặn bởi *trần của metric*: đa số câu chỉ có một trang golden nên Precision@5 chủ yếu tối đa ≈ 0.2 — không phản ánh truy hồi kém. **Faithfulness, Answer-Relevance, Correctness** chấm bằng **LLM-as-judge** (Gemini tự chấm câu trả lời) nên đọc như *chỉ báo*, không tuyệt đối — riêng Faithfulness 1.0 nên hiểu là “rất ít bịa trên bộ này”. **Thời gian ingest** đo trên *mẫu 2 tài liệu*: tài liệu text ≈7.5 s, còn PDF slide nhiều ảnh tốn ≈255 s do **OCR Gemini từng trang** chiếm phần lớn thời gian.

X.1. Đánh giá LightRAG/KG riêng (A/B trên câu hỏi quan hệ)

Trên 28 câu *đơn bước* ở trên, agent ADK tự chọn hầu như không gọi đường đồ thị (chủ yếu dùng search_knowledge_base), nên KG không thể hiện trong Bảng 4. Lợi ích của KG nằm ở câu hỏi *quan hệ/đa bước*, vì vậy nhóm soạn riêng **8 câu hỏi quan hệ** (nổi khái niệm giữa các tài liệu — ví dụ vai trò ↔ sự kiện trong Scrum, normalization ↔ CRUD) rồi chạy A/B *bật vs tắt* KG (cùng agent, chỉ khác đường đồ thị).

Phát hiện và sửa một lỗi runtime. Ban đầu KG trả rỗng ngay cả khi bật: tool truy vấn KG gọi

`asyncio.run()` trong khi runner của agent đã giữ một event loop đang chạy (lỗi “*cannot be called from a running event loop*”, bị cơ chế fail-safe nuốt). Nhóm đã sửa bằng cách chạy truy vấn KG trong một *luồng riêng*; sau sửa, KG trả về ~97K ký tự ngữ cảnh thực thể/quan hệ và đường đồ thị mới thực sự hoạt động trong luồng chat.

Bảng 6: A/B **LightRAG/KG** trên 8 câu hỏi quan hệ (cùng agent, chỉ bật/tắt đường đồ thị; chấm bằng LLM-as-judge).

Chỉ số (8 câu quan hệ)	KG tắt	KG bật
Correctness (so đáp án mẫu)	0.691	0.938
Answer Relevance	0.950	1.000
Faithfulness (bám nguồn)	0.841	0.694

Nhận xét A/B. Trên nhóm câu quan hệ, *bật* KG giúp **Correctness** tăng rõ (0.691 → 0.938) và Answer-Relevance (0.950 → 1.0) — ngữ cảnh thực thể/quan hệ bổ sung giúp trả lời sát đáp án hơn. Ngược lại **Faithfulness** giảm (0.841 → 0.694): kết quả KG *không kèm citation* nên phần bám-nguồn-theo-trích-dẫn bị chấm thấp hơn — đúng với hạn chế đã nêu của đường KG. **Lưu ý trung thực:** mẫu rất nhỏ ($n = 8$) và agent chỉ *tự gọi* KG ở 2/8 câu (các câu còn lại vẫn dùng đường chunk như nhau), cộng tính ngẫu nhiên của LLM — nên kết quả mang tính *gợi ý*, chưa phải kết luận mạnh. Hướng tiếp theo: tinh chỉnh để agent chủ động dùng KG cho câu quan hệ, và gắn citation cho kết quả KG.

XI. Phân tích lỗi và hạn chế

- **Truy xuất sai/thiếu:** chunk cắt ngang ý (do `chunk_size=800`); embedding chưa tối ưu tiếng Việt; câu hỏi mơ hồ. *Khắc phục:* cải thiện chunking, thử embedding tiếng Việt, hiện thực query-rewrite.
- **OCR yếu:** trang scan mờ, bảng/công thức → text rác làm nhiễu kho. *Khắc phục:* chỉnh ngưỡng OCR, hậu kiểm chất lượng text.
- **Hallucination / lệch nguồn:** LLM bịa khi context không đủ. *Khắc phục:* ràng buộc prompt “chỉ trả lời theo nguồn”, dùng rerank threshold / ngưỡng cosine, trả “không tìm thấy”.
- **Hội thoại:** LTM/history nạp sai ngữ cảnh, mất mạch khi câu hỏi nối tiếp. *Khắc phục:* tinh chỉnh `CHAT_LTM_MIN_SCORE`, số lượt lịch sử.
- **Hệ thống:** độ trễ cao khi tài liệu lớn; chi phí gọi LLM; mất cân bằng số tài liệu giữa chủ đề.

Hạn chế hiện tại. (1) **Query rewriting** mới khai báo cấu hình, *chưa* hiện thực trong code. (2) **Rerank** cần một HTTP endpoint cross-encoder cắm ngoài; nếu để trống thì bỏ qua. (3) **LightRAG/Knowledge Graph** cấu hình *bật* (`LIGHTRAG_ENABLED=true`) và **fail-safe** (lỗi

index/truy vấn KG bị nuốt, log, không raise — không bao giờ làm tài liệu FAILED; KG lỗi thì `search_knowledge_graph` trả rỗng, agent vẫn trả lời qua `search_knowledge_base`). Đánh đổi khi bật: cần dựng **PostgreSQL + Neo4j** và **driver pgvector** (không có sẵn trong stack mặc định đang chạy), cùng chi phí LLM trích thực thể lúc ingest; kết quả KG không kèm metadata nên *không sinh citation* (kéo Faithfulness xuống khi agent dùng KG — xem mục X.1.); chất lượng phụ thuộc bước trích thực thể. Trong thực nghiệm này KG đã được dựng và nạp đủ 9 tài liệu (**4407 thực thể / 6258 quan hệ**); nhóm **phát hiện và sửa** một lỗi runtime (`asyncio.run` trong event loop của agent) khiến KG trả rỗng trong luồng chat, sau đó chạy A/B trên 8 câu quan hệ (KG cải thiện Correctness 0.691 → 0.938, mục X.1.). Tuy vậy agent vẫn chỉ *tự gọi* KG ở số ít câu (2/8) — tinh chỉnh tool-routing là hướng tiếp theo. (4) `TOP_K/TOP_N/RRF_K` hardcoded trong code, chưa hot-tunable qua `.env`.

XII. Kết luận và hướng phát triển

Kết luận. Project chatbot giải quyết bài toán *retrieval-augmented question answering* trên tài liệu khóa học bằng một thành phần **chạy được**: pipeline ingest (trích/OCR → chunk + contextual header → embed 768-d → index OpenSearch HNSW), truy xuất **hybrid BM25 + kNN** hợp nhất bằng **RRF (k=60)** kèm lọc sần cosine (rerank cross-encoder cắm sẵn, fail-safe, chưa bật do thiếu endpoint), và sinh câu trả lời **streaming SSE** bằng **Gemini qua Google ADK**; ngoài ra một **đường truy hồi thứ hai LightRAG/KG** [1] (thực thể/quan hệ trên PostgreSQL/Neo4j, tool `search_knowledge_graph` mode mix) đã được hiện thực và cấu hình bật, fail-safe, thiết kế chạy hỗ trợ cho hybrid khi có hạ tầng KG (trên bộ test đơn bước agent chưa gọi tới, 0/28). Module tổ chức nhiều lớp, chạy nền bằng Celery; mọi tham số then chốt là *số thật* từ mã nguồn.

Hướng phát triển. Định hướng chính là *giảm phụ thuộc dịch vụ đám mây* (Gemini, Google ADK) và chuyển dần sang **tự host bằng mã nguồn mở** — để chủ động chi phí, dữ liệu và khả năng tinh chỉnh — theo lộ trình kiến thức của khóa học. Bốn nhánh migration:

- **Đọc tài liệu / OCR** (`pypdf` + *Gemini OCR* → OCR tự host): thêm OCR cho PDF scan/ảnh (Tesseract; hoặc YOLOv10 + CNN/Transformer), rồi chuẩn hoá & làm sạch text (tách đoạn, xử lý bảng, NER) và đối chiếu chất lượng trước khi thay hẳn Gemini.
- **Embedding** (*Gemini Embedding* → Ollama `nomic-embed-text 768d`): **đã tích hợp trong mã** (chọn provider qua `EMBEDDING_PROVIDER=ollama`), *chưa bật mặc định*; tiếp theo benchmark so với Gemini và tối ưu (batch, GPU, quantize).
- **LLM sinh câu trả lời** (*Gemini API* → Ollama `qwen2.5 / vLLM`): provider Ollama **đã tích hợp** qua ADK/LiteLlm (`LLM_PROVIDER=ollama`), *chưa bật mặc định*; tiếp theo serving bằng **vLLM**, nén model (quantization, ONNX/TensorRT) và đánh giá độ chính xác/an toàn trước khi thay hẳn.
- **Điều phối agent** (*Google ADK* → Python thuần): tự code luồng RAG *retrieve* → *ghép prompt* → *generate* với Tool Calling & ReAct, mở rộng **Multi-agent + MCP** khi cần, kèm logging & đánh giá pipeline (LLMOps).

Song song, **cải thiện lõi RAG** (độc lập với việc tự host): hiện thực **query-rewrite** (đã có cấu hình) để tăng recall; triển khai endpoint **rerank** `bge-reranker-v2-m3` và embedding tốt hơn

cho **tiếng Việt**; **đánh giá định lượng LightRAG/KG** cho câu hỏi đa bước và tinh chỉnh tool-routing để agent chủ động dùng KG [1], [6]; trích **citations** rõ trong câu trả lời, cache câu hỏi phổ biến, đánh giá tự động liên tục, tối ưu **TTFT**.

XIII. Tài liệu tham khảo

- [1] HKUDS. “LightRAG: Simple and fast retrieval-augmented generation (github)”, Accessed: Jun. 28, 2026. [Online]. Available: <https://github.com/HKUDS/LightRAG>
- [2] Neo4j. “Neo4j graph database documentation”, Accessed: Jun. 28, 2026. [Online]. Available: <https://neo4j.com/docs/>
- [3] pgvector. “Pgvector: Open-source vector similarity search for PostgreSQL”, Accessed: Jun. 28, 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [4] LangChain. “Recursively split by character (RecursiveCharacterTextSplitter)”, Accessed: Jun. 28, 2026. [Online]. Available: https://python.langchain.com/docs/how_to/recursive_text_splitter/
- [5] OpenSearch Project. “Opensearch: Vector search and k-NN”, Accessed: Jun. 27, 2026. [Online]. Available: <https://opensearch.org/docs/latest/search-plugins/knn/index/>
- [6] Microsoft. “GraphRAG documentation”, Accessed: Jun. 28, 2026. [Online]. Available: <https://microsoft.github.io/graphrag/>
- [7] Google. “Agent development kit (ADK) documentation”, Accessed: Jun. 27, 2026. [Online]. Available: <https://google.github.io/adk-docs/>
- [8] Django Software Foundation. “Django: The web framework for perfectionists with deadlines”, Accessed: Jun. 27, 2026. [Online]. Available: <https://www.djangoproject.com/>
- [9] Celery Project. “Celery: Distributed task queue”, Accessed: Jun. 27, 2026. [Online]. Available: <https://docs.celeryq.dev/>
- [10] Google. “Gemini API documentation”, Accessed: Jun. 28, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>

Phụ lục

1. **AIVIETNAM:** Thông tin về AIVIETNAM có thể đọc thêm tại [đây](#).
2. **GitHub Repository:** github.com/AIVIETNAM-AIO-NG-LINH/aio-2026.
3. **Demo App / Website:** aio-it.dev.
4. **Demo Video:** [RAG Chatbot — Overfitting Club \(Demo\)](#).
5. **Dataset / Tài liệu khóa học:** 9 tài liệu khóa học AIO 2026 (Python cơ bản: vòng lặp & rẽ nhánh/chatbot luật; SQL & ERD; Coding Methodology; Image & Video Data; Teamwork Tools; hướng dẫn làm video; chính sách bảo mật) — nạp thành 782 chunk trên rag-index.